Cryptocurrency Price Prediction System

Automated Pipeline from Data Collection to Model Training and Deployment

Machine Learning Project Report

---

1. Project Title & Motivation

Title

Intelligent Cryptocurrency Q&A and Price Prediction System

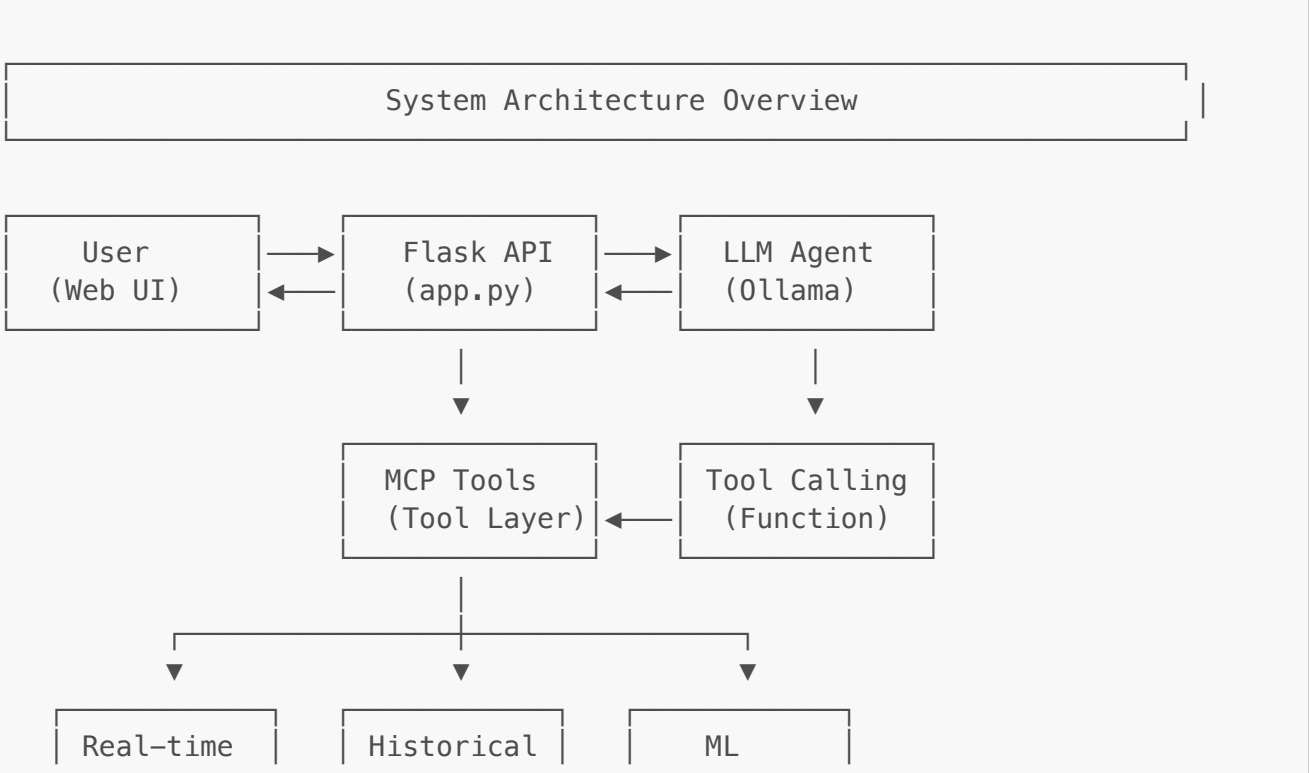
- An end-to-end MLOps project combining NLP and Deep Learning

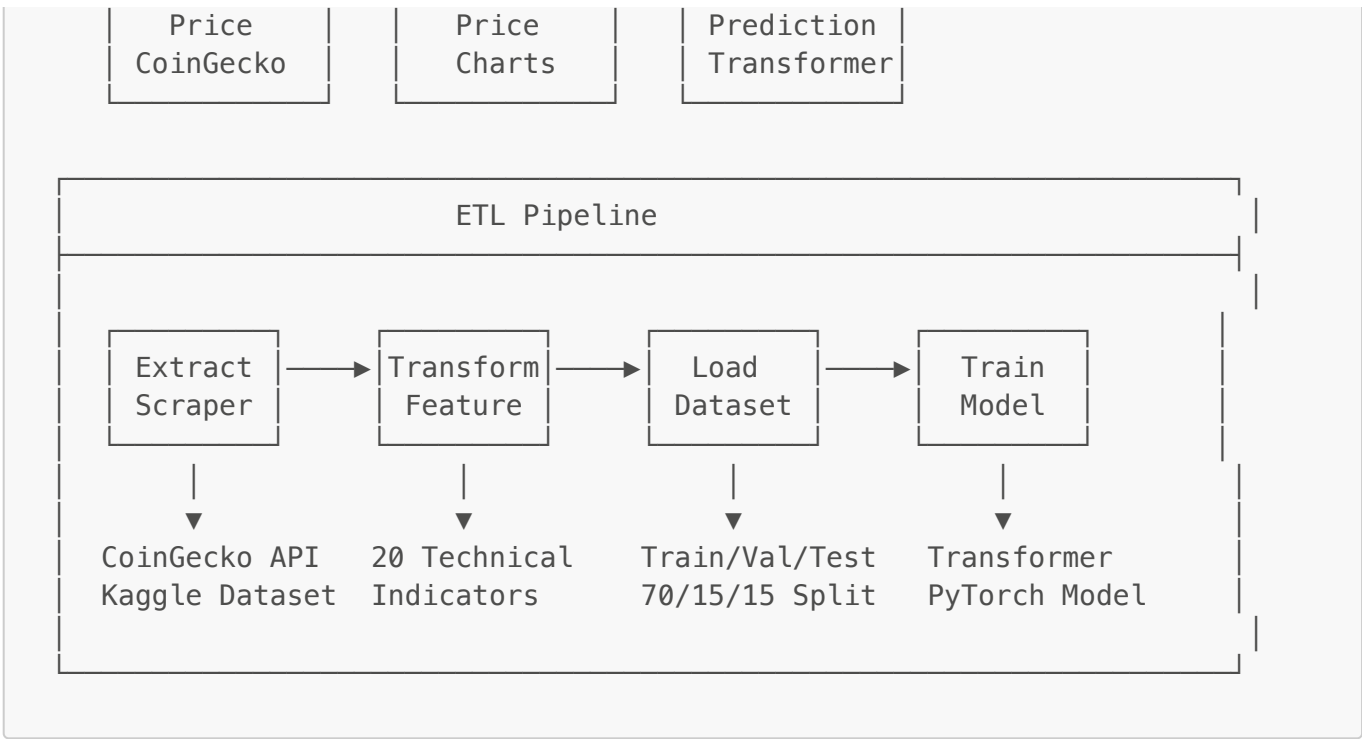
Motivation

1. **Market Demand**: Cryptocurrency markets are highly volatile; investors need real-time intelligent analysis tools
2. **Technical Integration**: Implement a complete MLOps workflow from data collection, cleaning, training to deployment
3. **Learning Objectives**:
    - Build ETL Pipeline for automated data processing
    - Train Transformer deep learning model
    - Deploy RESTful API using Flask
    - Integrate LLM (Ollama) for natural language Q&A

---

2. System Architecture (ETL / Pipeline)





Project Directory Structure

```
ML-Crypto-QA-Interface/
├── api/                    # Flask API Service
│   ├── app.py             # Main Application
│   └── static/            # Frontend UI
├── data/                   # Data Layer
│   ├── raw/               # Raw Data (Kaggle)
│   └── scrapers/          # Data Scrapers (CoinGecko)
├── etl/                    # ETL Pipeline
│   ├── extract.py         # Data Extraction
│   └── transform.py       # Feature Engineering
├── models/                 # Model Layer
│   ├── transformer/       # Transformer Model
│   │   ├── model.py      # Model Architecture
│   │   ├── train.py      # Training Script
│   │   └── predictor.py  # Predictor
│   └── artifacts/         # Model Files
├── orchestrator/           # LLM Agent
│   └── agent.py           # ReAct Pattern Agent
├── mcp/                    # MCP Tool Definitions
│   └── tools.py           # Tool Implementations
└── evaluation/             # Evaluation Results
    └── results/           # Training Reports
```

---

3. Data Layer: DataOps

3.1 Data Scraping (Extract)

Data Sources

Source	Purpose	Data Volume
CoinGecko API	Real-time prices, historical trends	Real-time
Kaggle Dataset	Model training	4,371 records (BTC)

CoinGecko API Integration (`data/scrapers/coincap_client.py`)

```
# Supported Cryptocurrencies
SYMBOL_MAP = {
    "BTC": "bitcoin",
    "ETH": "ethereum",
    "SOL": "solana"
}

# API Endpoints
- /simple/price             # Real-time price
- /coins/{id}/market_chart  # Historical trends
```

3.2 Data Cleaning & Feature Engineering (Transform)

20 Technical Indicator Features

Category	Feature Names	Description
Returns	return_1d, 3d, 5d, 10d	Multi-scale returns
Returns	log_return_1d	Log returns
RSI	rsi_7, rsi_14, rsi_21	Relative Strength Index
Volatility	volatility_5d, 10d, 20d	Price volatility
MA	ma_ratio_5, ma_ratio_20	Moving average ratio
MA	ma_cross_5_20	MA crossover signal
MACD	macd_hist	MACD histogram
BB	bb_width, bb_position	Bollinger Bands
Other	high_low_ratio	High-low ratio
Other	close_position	Close price position
Other	volume_ratio	Volume ratio

Data Processing Pipeline

```
# 1. Load raw data
df = load_kaggle_data("BTC")  # 4,371 records

# 2. Calculate technical indicators
```

```
df = add_features(df)  # 20 features

# 3. Normalization
features = normalize(features)  # Z-score

# 4. Split dataset
Train: 70% (3,029 records)
Val:   15% (626 records)
Test:  15% (626 records)
```

---

4. Model Layer: MLOps

4.1 Model Architecture: Transformer

```
┌─────────────────────────────────────────┐
│          CryptoTransformer              │
├─────────────────────────────────────────┤
│  Input: [batch, 30, 20]                 │
│         (seq_len=30, features=20)       │
├─────────────────────────────────────────┤
│  1. Input Projection                    │
│     Linear(20 → 64) + LayerNorm + GELU  │
├─────────────────────────────────────────┤
│  2. Positional Encoding                 │
│     Sinusoidal position encoding        │
├─────────────────────────────────────────┤
│  3. Transformer Encoder (x2 layers)     │
│     - Multi-Head Attention (4 heads)    │
│     - Feed Forward (128 dim)            │
│     - Dropout (0.4)                     │
├─────────────────────────────────────────┤
│  4. Attention Pooling                   │
│     Weighted average of all timesteps   │
├─────────────────────────────────────────┤
│  5. Classifier                          │
│     Linear → LayerNorm → GELU →         │
│     Linear → Output [batch, 2]          │
├─────────────────────────────────────────┤
│  Output: UP / DOWN probabilities        │
└─────────────────────────────────────────┘
```

Model Parameters

Parameter	Value
Input Dimension	20
Model Dimension	64

Parameter	Value
Attention Heads	4
Encoder Layers	2
Dropout	0.4
Total Parameters	~50K

4.2 Training Configuration

```
config = {
    "epochs": 200,
    "batch_size": 32,
    "learning_rate": 0.0002,
    "weight_decay": 0.1,      # L2 regularization
    "label_smoothing": 0.1,   # Label smoothing
    "warmup_epochs": 15,      # Learning rate warmup
    "patience": 30            # Early Stopping
}
```

Training Techniques

- AdamW optimizer + weight decay
- Cosine Annealing learning rate schedule
- Gradient clipping (max_norm=1.0)
- Class weight balancing

4.3 Flask API Deployment

 Flask Server Startup

```
# api/app.py - Main Endpoints

@app.route('/api/chat', methods=['POST'])
def chat():
    """Intelligent Q&A - Natural language queries"""
    # 1. Parse user message
    # 2. Determine intent (price/predict/chart)
    # 3. Call corresponding tool
    # 4. Return result

@app.route('/api/predict/<symbol>')
def predict(symbol):
    """Price Prediction API"""
    # Use Transformer model for prediction

@app.route('/api/price/<symbol>')
def get_price(symbol):
```

```
    """Real-time Price API"""
    # Call CoinGecko API
```

API Features

Endpoint	Function	Example
POST /api/chat	Natural language Q&A	"Will BTC go up?"
GET /api/predict/BTC	Price prediction	Returns UP/DOWN
GET /api/price/BTC	Real-time price	Returns $97,250
GET /api/history/BTC	Historical trends	Returns chart data

預測 BTC 明天的價格走勢

根據系統使用 **LightGBM** 模型（模型內部採用了 transformer 方式）對 BTC（比特幣）進行的最新預測，結果如下：

項目	數值
預測方向	**DOWN（下跌）**
預測概率	**48.23%**
信心水平	**51.77%**（屬於「低」）
主要特徵	- 前一日報酬率（return*1d*）：**+2.78%** - *14 天相對強弱指標（RSI*14）：**60.49** - 7 天波動率（volatility_7d）：**3.98%**
預測時間	2025-12-22 22:33:52（UTC）

1. 何謂「預測方向：DOWN」？

模型判斷明天比特幣的收盤價較今天可能會**下跌**。然而，預測概率僅為 48.23%，說明模型對「下跌」的信心並不高，事實上上漲的機率稍高（約 51.77%）。

2. 為什麼信心水平只有「低」？

- **預測概率接近 50%**：模型的判斷幾乎是「持平」的狀態，沒有明顯的偏向。
- **特徵解讀**：
- **return_1d = +2.78%**：過去 24 小時內比特幣已經上漲了近 3%，市場近期呈現上行勢頭。
- **RSI_14 = 60.49**：RSI 超過 60，已進入「超買」區間（通常以 70 為超買臨界），暗示短期內可能出現回調壓力。
- **volatility_7d = 3.98%**：過去一週的波動率屬於中等偏高，意味著價格仍在頻繁波動，容易被外部訊息或資金流向推動方向改變。

綜合上述，模型在「下跌」與「持平」之間搖擺，且因近期已經有不小的漲幅且 RSI 接近超買，系統給出「下跌」的提示，但仍保留了相當大的不確定性。

3. 投資者可以怎麼看待這個訊號？

- **保持謹慎**
  - 低信心水平意味著預測不夠穩固，建議不要僅憑此訊號做出大額交易決策。
- **觀察關鍵技術指標**
  - 若 RSI 繼續上升突破 70，回調的概率會增大。
  - 若價格在短期內跌破近期支撐（例如前高或重要均線），則可能驗證模型的「下跌」判斷。
- **關注市場基本面**
  - 大型機構資金流入/流出、監管消息或宏觀經濟變化（如美國利率、美元走勢）都可能在短期內顛覆技術指標的預測。
- **風險管理**
  - 若決定持有或加碼，請設定止損位（如 3-5%）以防突發的價格逆轉。
  - 若想捕捉可能的回調，可考慮在 2-3% 的回撤區域佈局空單或對沖部位。

---

5. Optional: Advanced Features

5.1 LLMOps - LLM Agent Integration

ReAct Pattern (Reasoning + Acting)

```
User: "What's the current BTC price? Will it go up tomorrow?"
     |
     ▼
```

```
┌─────────────────────────────────────┐
│  LLM Agent (Ollama - llama3.2)      │
│  1. Understand user intent          │
│  2. Select tool: get_current_price  │
│  3. Select tool: predict_price      │
│  4. Interpret results               │
│  5. Respond in natural language     │
└─────────────────────────────────────┘
     │
     ▼
"BTC current price is $97,250, up 2.15% in 24h.
 Based on Transformer model prediction, there's
 a 64.53% probability of rising tomorrow,
 with medium confidence level."
```

MCP Tools Definition (`mcp/tools.py`)

```python
TOOLS = [
    "get_current_price",      # Real-time price
    "get_price_history",      # Historical trends
    "predict_price",          # ML prediction
    "get_technical_analysis", # Technical analysis
    "analyze_sentiment"       # Sentiment analysis
]
```

5.2 Docker Support

```dockerfile
# docker/Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5001
CMD ["python", "-m", "api.app"]
```

---

6. Model Evaluation & Results

6.1 Training Results

Metric	Value	Description
Best Validation Accuracy	**57.99%**	Achieved at Epoch 47
Test Accuracy	**54.63%**	Better than random guess (50%)
Test Precision	59.36%	Accuracy for "UP" predictions

Metric	Value	Description
Test F1 Score	43.87%	Combined evaluation metric
Training Time	100.69 sec	Early Stopping at epoch 77

Results Analysis

- Financial market prediction is inherently challenging; 55-60% accuracy is considered valuable in academic research
- Model uses Early Stopping to prevent overfitting, achieving best validation at Epoch 47
- Label Smoothing (0.1) and high Dropout (0.4) enhance generalization

6.2 Prediction Demo

(Prediction Result Example)

```
{
  "symbol": "BTC",
  "prediction": "UP",
  "confidence": 0.6453,
  "probabilities": {
    "UP": 0.6453,
    "DOWN": 0.3547
  },
  "model": "Transformer",
  "confidence_level": "Medium"
}
```

6.3 Lessons Learned

What I Learned

1. **Complete MLOps Workflow**
    - From data collection, cleaning, feature engineering to model training and deployment
    - Understanding the importance of ETL Pipeline
2. **Transformer Architecture**
    - Applied Transformer (commonly used in NLP) to time series prediction
    - Understanding how Attention mechanism captures long-term dependencies
3. **API Design & Deployment**
    - Flask RESTful API design
    - Frontend-backend integration and CORS handling
4. **LLM Agent Integration**
    - Function Calling / Tool Use mechanism

- ReAct pattern implementation

6.4 Challenges & Solutions

Challenge	Solution
CoinCap API discontinued	Migrated to CoinGecko API
SSL certificate issues (macOS)	Custom SSL Context
Model overfitting	Increased Dropout, Label Smoothing
Feature dimension mismatch	Unified feature processing for training and prediction
LLM doesn't support Function Calling	Implemented fallback rule matching

6.5 Future Improvements

1. **Model Enhancement**
   - Add more features (news sentiment, on-chain data)
   - Try other architectures (LSTM, Temporal Fusion Transformer)
2. **System Expansion**
   - Support more cryptocurrencies
   - Add automated training scheduling
   - Integrate MLflow for experiment tracking
3. **Deployment Optimization**
   - Use Docker Compose for complete deployment
   - Add monitoring and alerting mechanisms

Technology Stack Overview

Category	Technology
Programming Language	Python 3.11
Deep Learning	PyTorch
Web Framework	Flask
Frontend	HTML/CSS/JavaScript, Chart.js
LLM	Ollama (llama3.2)
Data Processing	Pandas, NumPy
API	CoinGecko
Version Control	Git
Containerization	Docker

Q&A

Thank you for listening!